

UPDATED FOR 2026 INTERVIEWS

Top 20 SQL Interview Questions & Answers

Everything You Need to Crack SQL Interviews at Product-Based Companies

Target Audience

Freshers | Data Engineers | Backend Devs | MAANG Aspirants

Prepared by AlgoTutor

Elevating Tech Careers Worldwide

Welcome Note

Why SQL Matters in 2026

Despite the rise of NoSQL and massive distributed systems, standard SQL remains the absolute backbone of data querying, engineering, and backend architecture. Whether you're interviewing at Google for a Software Engineering role or at Stripe for Data Analytics, your ability to write optimized, highly performant SQL queries is non-negotiable.

How to Use This Handbook

We designed this guide to bridge the gap between academic SQL and production-level SQL. Do not just memorize the answers. Pay close attention to the **"Real-World Use Case"** and **"Performance Considerations"** sections. Interviewers at top-tier product companies test your practical implementation knowledge, not just syntax.

- **Read sequentially:** Concepts build upon each other.
- **Review the Cheat Sheets:** Use them for last-minute revision before a screening round.
- **Practice:** Run the provided syntax on your local DB or SQL Fiddle.

Estimated Reading Time: ~2.5 Hours

Pace yourself. Master 5 questions a day and utilize the quick revision blocks to cement your knowledge.

QUESTION 1

Difference between DELETE, TRUNCATE, and DROP?

Difficulty: Beginner

Asked in: Amazon, Flipkart, TCS, Infosys

WHY INTERVIEWERS ASK THIS

Interviewers ask this to test your understanding of DML vs DDL commands, transaction logging, and database memory management.

DETAILED ANSWER

DELETE is a DML command used to remove specific rows based on a WHERE clause. It logs each deletion and can be rolled back. TRUNCATE is a DDL command that removes all rows from a table by deallocating the data pages, making it much faster but it cannot be rolled back in some databases. DROP is a DDL command that completely removes the table structure, data, privileges, and indexes from the database.

SYNTAX OVERVIEW

```
DELETE FROM table_name WHERE condition;  
TRUNCATE TABLE table_name;  
DROP TABLE table_name;
```

REAL EXAMPLE

```
-- Delete specific row  
DELETE FROM Employees WHERE emp_id = 101;  
  
-- Empty the table  
TRUNCATE TABLE Employees;  
  
-- Destroy the table entirely  
DROP TABLE Employees;
```

Output:

```
DELETE: 1 row affected.  
TRUNCATE: Table emptied, 0 rows affected.  
DROP: Table no longer exists.
```

STEP-BY-STEP EXECUTION

- 1. Identify if you need to remove specific data (DELETE) or all data (TRUNCATE).
- 2. Use DROP only when the table structure is no longer needed.
- 3. Ensure you have backups before executing TRUNCATE or DROP.

Interview Tip

Always mention the difference in Transaction Logging (DELETE is row-level logged, TRUNCATE is page-level logged).

Common Mistake

Forgetting the WHERE clause in a DELETE statement, which accidentally removes all data (though much slower than TRUNCATE).

ADVANCED CONCEPTS

Real-World Use Case: Use DELETE when a user deletes their account. Use TRUNCATE to clear out daily staging tables in ETL processes. Use DROP when a temporary table is no longer needed after processing.

Best Practices: Always test DELETE statements as SELECT statements first to ensure you are targeting the correct rows.

Performance Considerations: TRUNCATE is significantly faster than DELETE without a WHERE clause because it doesn't write each deleted row to the transaction log.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can you roll back a TRUNCATE command?
- Does TRUNCATE reset the identity column/auto-increment?
- What permissions are required for DROP?

QUESTION 2

Explain INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN.

Difficulty: Beginner

Asked in: Google, Uber, Microsoft, Swiggy

WHY INTERVIEWERS ASK THIS

Joins are the most fundamental aspect of relational databases. Interviewers want to know if you can correctly retrieve related data across multiple tables without data loss or duplication.

DETAILED ANSWER

INNER JOIN returns only the rows that have matching values in both tables. LEFT JOIN returns all rows from the left table, and matching rows from the right table (NULL if no match). RIGHT JOIN returns all rows from the right table, and matching rows from the left. FULL JOIN returns all rows when there is a match in either left or right table.

SYNTAX OVERVIEW

```
SELECT columns
FROM table1
[INNER | LEFT | RIGHT | FULL] JOIN table2
ON table1.column = table2.column;
```

REAL EXAMPLE

```
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
ON e.DeptID = d.DeptID;
```

Output:

Name	DepartmentName
John	IT
Jane	NULL

STEP-BY-STEP EXECUTION

- 1. Identify the primary table (usually the one in the FROM clause).
- 2. Determine which tables need to be linked.
- 3. Choose the appropriate join type based on whether you need unmatched records.
- 4. Specify the linking condition in the ON clause.

Interview Tip

Venn diagrams are great, but explain Joins in terms of 'driving tables' and data preservation to impress senior interviewers.

Common Mistake

Using a LEFT JOIN but putting a filter on the right table in the WHERE clause, which accidentally converts it into an INNER JOIN.

ADVANCED CONCEPTS

Real-World Use Case: LEFT JOIN is heavily used to find missing relationships, e.g., finding all Users who have NOT placed an Order (where OrderID is NULL).

Best Practices: Always use table aliases (e.g., 'e' for Employees) to make queries readable and prevent ambiguous column errors.

Performance Considerations: Ensure the columns used in the ON clause are indexed. Avoid joining on text or highly unoptimized data types.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- What happens if you join tables with duplicate keys?
- What is a CROSS JOIN?
- How do you optimize a slow JOIN?

QUESTION 3

Difference between WHERE and HAVING clause?

Difficulty: Intermediate

Asked in: Oracle, Adobe, Salesforce, Atlassian

WHY INTERVIEWERS ASK THIS

Tests the understanding of SQL's logical execution order. Many candidates confuse filtering raw data vs filtering aggregated data.

DETAILED ANSWER

WHERE is used to filter rows before any grouping or aggregation takes place. It cannot be used with aggregate functions (like SUM, COUNT). HAVING is used to filter groups after the GROUP BY clause has been applied and can evaluate aggregate functions.

SYNTAX OVERVIEW

```
SELECT column1, SUM(column2)
FROM table
WHERE condition (Row-level)
GROUP BY column1
HAVING condition (Group-level);
```

REAL EXAMPLE

```
SELECT Department, COUNT(EmpID) as TotalStaff
FROM Employees
WHERE IsActive = 1
GROUP BY Department
HAVING COUNT(EmpID) > 5;
```

Output:

```
Department | TotalStaff
-----
Sales | 8
Engineering | 12
```

STEP-BY-STEP EXECUTION

- 1. The WHERE clause filters out inactive employees first.

- 2. The remaining rows are grouped by Department.
- 3. COUNT is calculated for each group.
- 4. HAVING filters out departments with 5 or fewer active employees.

Interview Tip

Always mention the Logical Order of Processing: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT.

Common Mistake

Trying to use aggregate functions in the WHERE clause (e.g., WHERE SUM(Sales) > 1000).

ADVANCED CONCEPTS

Real-World Use Case: Generating reports where you only want to highlight high-performing segments (e.g., Regions where total sales exceed \$100,000).

Best Practices: Filter as much data as possible in the WHERE clause before grouping to save computation power.

Performance Considerations: Filtering in WHERE utilizes indexes. Filtering in HAVING usually scans in-memory grouped data, which is slower if the initial dataset is huge.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can you use HAVING without GROUP BY?
- Does WHERE improve performance more than HAVING?

QUESTION 4

Explain Primary Key, Unique Key, and Foreign Key.

Difficulty: Beginner

Asked in: Infosys, Wipro, TCS, Amazon

WHY INTERVIEWERS ASK THIS

This evaluates your understanding of database constraints, data integrity, and relational design principles.

DETAILED ANSWER

A Primary Key uniquely identifies each record in a table; it cannot contain NULLs and a table can have only one. A Unique Key also ensures uniqueness but allows one NULL value, and a table can have multiple. A Foreign Key is a field in one table that uniquely identifies a row of another table, establishing a relationship and enforcing referential integrity.

SYNTAX OVERVIEW

```
CREATE TABLE Users (  
  ID INT PRIMARY KEY,  
  Email VARCHAR(50) UNIQUE,  
  DeptID INT FOREIGN KEY REFERENCES Depts(ID)  
);
```

REAL EXAMPLE

```
-- Creating constraints  
ALTER TABLE Orders  
ADD CONSTRAINT FK_User  
FOREIGN KEY (UserID) REFERENCES Users(ID);
```

Output:

Constraint applied successfully. Insertions failing referential integrity will now throw an error.

STEP-BY-STEP EXECUTION

- 1. Assign Primary Keys to surrogate IDs or unique identifiers.
- 2. Apply Unique constraints to business keys (Emails, Usernames).

- 3. Map relationships using Foreign Keys.

Interview Tip

Explain that Primary Keys automatically create a Clustered Index (in most RDBMS), while Unique Keys create a Non-Clustered Index.

Common Mistake

Confusing a Foreign Key with a simple integer column. Without the actual constraint, referential integrity is not enforced by the engine.

ADVANCED CONCEPTS

Real-World Use Case: Primary Key on UserID. Unique Key on Email to prevent duplicate sign-ups. Foreign Key on Order.UserID to ensure orders are placed by valid users.

Best Practices: Name your constraints explicitly (e.g., PK_Users, FK_Orders_Users) so they are easier to manage and drop later.

Performance Considerations: Always index Foreign Key columns, as they are frequently used in JOINS.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can a table have multiple foreign keys?
- What is a Composite Primary Key?
- What happens on DELETE CASCADE?

QUESTION 5

Difference between ROW_NUMBER(), RANK(), and DENSE_RANK()?

Difficulty: Advanced

Asked in: Meta, Netflix, Amazon, Uber

WHY INTERVIEWERS ASK THIS

Window functions are critical for advanced analytics. Interviewers use this to see if you can perform complex data rankings and solve top-N per group problems.

DETAILED ANSWER

ROW_NUMBER() assigns a unique, sequential integer to rows. RANK() assigns the same rank to identical values but skips the next rank numbers (e.g., 1, 2, 2, 4). DENSE_RANK() assigns the same rank to identical values but does NOT skip numbers (e.g., 1, 2, 2, 3).

SYNTAX OVERVIEW

```
SELECT column,  
    ROW_NUMBER() OVER(ORDER BY column) as RN,  
    RANK() OVER(ORDER BY column) as Rnk,  
    DENSE_RANK() OVER(ORDER BY column) as DRnk  
FROM table;
```

REAL EXAMPLE

```
SELECT Score,  
    ROW_NUMBER() OVER(ORDER BY Score DESC) as RN,  
    RANK() OVER(ORDER BY Score DESC) as Rnk,  
    DENSE_RANK() OVER(ORDER BY Score DESC) as DRnk  
FROM ExamScores;
```

Output:

Score	RN	Rnk	DRnk
95	1	1	1
90	2	2	2
90	3	2	2
85	4	4	3

STEP-BY-STEP EXECUTION

- 1. The OVER clause defines the window (sorting by Score).
- 2. ROW_NUMBER blindly increments.
- 3. RANK gives ties the same value and jumps.
- 4. DENSE_RANK gives ties the same value but continues sequentially.

Interview Tip

If asked to find the 2nd highest salary, instantly mention DENSE_RANK().

Common Mistake

Using RANK() for Nth highest problems, which fails if multiple people share the 1st highest value.

ADVANCED CONCEPTS

Real-World Use Case: Finding the 'Nth Highest Salary' (Use DENSE_RANK to handle ties properly). Deduplicating rows (Use ROW_NUMBER with PARTITION BY).

Best Practices: Always specify an ORDER BY within the OVER() clause when using ranking functions, otherwise results are unpredictable.

Performance Considerations: Window functions require sorting, which can be memory intensive. Ensure indexes exist on the ORDER BY / PARTITION BY columns.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- How do you use PARTITION BY with these functions?
- How can ROW_NUMBER() be used to remove duplicates?

Quick Revision: Questions 1 to 5

Before moving forward, ensure you have mastered these key concepts. Interviewers often link these topics together during deep-dive rounds.

Memory Trick

Always map SQL concepts to real-life analogues. Joins are like Venn diagrams (but with tables). Indexes are like the glossary of a textbook. Transactions are like ATM withdrawals (all or nothing).

Interviewer's Checklist

- Did the candidate explain the 'Why' and not just the 'How'?
- Were performance and indexing mentioned voluntarily?
- Did they write syntax cleanly without prompt?

QUESTION 6

Clustered vs Non-Clustered Index?

Difficulty: Advanced

Asked in: Microsoft, Oracle, Salesforce, Apple

WHY INTERVIEWERS ASK THIS

Understanding indexes is the difference between a query taking 1 second vs 1 hour. This tests your grasp on database architecture and optimization.

DETAILED ANSWER

A Clustered Index determines the physical order of data rows in a table (like a phone book sorted by last name); thus, a table can only have one. A Non-Clustered Index creates a separate structure that contains the indexed columns and a pointer to the actual data row (like an index at the back of a book); a table can have many.

SYNTAX OVERVIEW

```
CREATE CLUSTERED INDEX IX_EmpID ON Employees(EmpID);  
CREATE NONCLUSTERED INDEX IX_EmpName ON Employees(Name);
```

REAL EXAMPLE

```
-- Creating an index to speed up email lookups  
CREATE NONCLUSTERED INDEX idx_user_email  
ON Users (Email);
```

Output:

Index created. Subsequent SELECT queries filtering by Email will use an Index Seek instead of a Table Scan.

STEP-BY-STEP EXECUTION

- 1. The engine checks if an index exists for the WHERE clause.
- 2. For non-clustered, it traverses the B-Tree index, finds the pointer.
- 3. It uses the pointer (RowID or Clustered Key) to fetch the full row data.

Interview Tip

Mention B-Tree (Balanced Tree) architecture. Explain that primary keys create clustered indexes by default.

Common Mistake

Creating too many non-clustered indexes. Every index speeds up reads but slows down writes (INSERT/UPDATE/DELETE).

ADVANCED CONCEPTS

Real-World Use Case: Clustered index on ID columns (sequential). Non-clustered index on columns frequently used in WHERE, JOIN, or ORDER BY (e.g., Email, Status, CreatedDate).

Best Practices: Keep clustered index keys narrow, static, and ever-increasing (like an INT IDENTITY column).

Performance Considerations: Use non-clustered indexes for foreign keys to dramatically speed up JOIN operations.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- What is a Covering Index?
- What happens if you don't have a clustered index (Heap)?
- What is index fragmentation?

QUESTION 7

CTE (Common Table Expression) vs Subquery?

Difficulty: Intermediate

Asked in: Stripe, Airbnb, Atlassian

WHY INTERVIEWERS ASK THIS

Interviewers want to see if you write clean, maintainable SQL code. CTEs are the modern standard for structuring complex queries.

DETAILED ANSWER

A Subquery is a query nested inside another query. A CTE is a temporary named result set defined using the WITH clause. CTEs are generally more readable, can be referenced multiple times within the same query, and support recursive queries, whereas subqueries can become deeply nested and harder to read.

SYNTAX OVERVIEW

```
WITH CTE_Name AS (  
    SELECT * FROM Table  
)  
SELECT * FROM CTE_Name;
```

REAL EXAMPLE

```
WITH HighEarnings AS (  
    SELECT EmpID, Salary FROM Employees WHERE Salary > 100000  
)  
SELECT * FROM HighEarnings WHERE EmpID IN (SELECT EmpID FROM Managers);
```

Output:

Returns all managers who also earn over 100,000.

STEP-BY-STEP EXECUTION

- 1. Define the temporary data set in the WITH clause.
- 2. The CTE is stored in memory during the query execution.
- 3. The main SELECT statement consumes the CTE data.

Interview Tip

If an interviewer asks how to query a self-referencing table (Manager-Employee), immediately say 'Recursive CTE'.

Common Mistake

Thinking CTEs are permanent or act like views. They only exist for the duration of the execution of the statement.

ADVANCED CONCEPTS

Real-World Use Case: Breaking down massive, 1000-line financial reporting queries into logical, sequential blocks. Traversing hierarchical data (like org charts) using Recursive CTEs.

Best Practices: Always use CTEs instead of derived tables (subqueries in the FROM clause) for better readability.

Performance Considerations: Performance between CTEs and standard subqueries is usually identical, as the query optimizer unpacks them the same way.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Are CTEs faster than Subqueries?
- How does a Recursive CTE work?
- Can you use multiple CTEs in one statement?

QUESTION 8

GROUP BY and Aggregate Functions

Difficulty: Beginner

Asked in: Amazon, Flipkart, TCS, Infosys

WHY INTERVIEWERS ASK THIS

Interviewers ask this to test your understanding of DML vs DDL commands, transaction logging, and database memory management.

DETAILED ANSWER

DELETE is a DML command used to remove specific rows based on a WHERE clause. It logs each deletion and can be rolled back. TRUNCATE is a DDL command that removes all rows from a table by deallocating the data pages, making it much faster but it cannot be rolled back in some databases. DROP is a DDL command that completely removes the table structure, data, privileges, and indexes from the database.

SYNTAX OVERVIEW

```
DELETE FROM table_name WHERE condition;  
TRUNCATE TABLE table_name;  
DROP TABLE table_name;
```

REAL EXAMPLE

```
-- Delete specific row  
DELETE FROM Employees WHERE emp_id = 101;  
  
-- Empty the table  
TRUNCATE TABLE Employees;  
  
-- Destroy the table entirely  
DROP TABLE Employees;
```

Output:

```
DELETE: 1 row affected.  
TRUNCATE: Table emptied, 0 rows affected.  
DROP: Table no longer exists.
```

STEP-BY-STEP EXECUTION

- 1. Identify if you need to remove specific data (DELETE) or all data (TRUNCATE).
- 2. Use DROP only when the table structure is no longer needed.
- 3. Ensure you have backups before executing TRUNCATE or DROP.

Interview Tip

Always mention the difference in Transaction Logging (DELETE is row-level logged, TRUNCATE is page-level logged).

Common Mistake

Forgetting the WHERE clause in a DELETE statement, which accidentally removes all data (though much slower than TRUNCATE).

ADVANCED CONCEPTS

Real-World Use Case: Use DELETE when a user deletes their account. Use TRUNCATE to clear out daily staging tables in ETL processes. Use DROP when a temporary table is no longer needed after processing.

Best Practices: Always test DELETE statements as SELECT statements first to ensure you are targeting the correct rows.

Performance Considerations: TRUNCATE is significantly faster than DELETE without a WHERE clause because it doesn't write each deleted row to the transaction log.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can you roll back a TRUNCATE command?
- Does TRUNCATE reset the identity column/auto-increment?
- What permissions are required for DROP?

QUESTION 9

UNION vs UNION ALL

Difficulty: Beginner

Asked in: Google, Uber, Microsoft, Swiggy

WHY INTERVIEWERS ASK THIS

Joins are the most fundamental aspect of relational databases. Interviewers want to know if you can correctly retrieve related data across multiple tables without data loss or duplication.

DETAILED ANSWER

INNER JOIN returns only the rows that have matching values in both tables. LEFT JOIN returns all rows from the left table, and matching rows from the right table (NULL if no match). RIGHT JOIN returns all rows from the right table, and matching rows from the left. FULL JOIN returns all rows when there is a match in either left or right table.

SYNTAX OVERVIEW

```
SELECT columns
FROM table1
[INNER | LEFT | RIGHT | FULL] JOIN table2
ON table1.column = table2.column;
```

REAL EXAMPLE

```
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
ON e.DeptID = d.DeptID;
```

Output:

```
Name | DepartmentName
-----
John | IT
Jane | NULL
```

STEP-BY-STEP EXECUTION

- 1. Identify the primary table (usually the one in the FROM clause).

- 2. Determine which tables need to be linked.
- 3. Choose the appropriate join type based on whether you need unmatched records.
- 4. Specify the linking condition in the ON clause.

Interview Tip

Venn diagrams are great, but explain Joins in terms of 'driving tables' and data preservation to impress senior interviewers.

Common Mistake

Using a LEFT JOIN but putting a filter on the right table in the WHERE clause, which accidentally converts it into an INNER JOIN.

ADVANCED CONCEPTS

Real-World Use Case: LEFT JOIN is heavily used to find missing relationships, e.g., finding all Users who have NOT placed an Order (where OrderID is NULL).

Best Practices: Always use table aliases (e.g., 'e' for Employees) to make queries readable and prevent ambiguous column errors.

Performance Considerations: Ensure the columns used in the ON clause are indexed. Avoid joining on text or highly unoptimized data types.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- What happens if you join tables with duplicate keys?
- What is a CROSS JOIN?
- How do you optimize a slow JOIN?

QUESTION 10

Stored Procedures vs Functions

Difficulty: Intermediate

Asked in: Oracle, Adobe, Salesforce, Atlassian

WHY INTERVIEWERS ASK THIS

Tests the understanding of SQL's logical execution order. Many candidates confuse filtering raw data vs filtering aggregated data.

DETAILED ANSWER

WHERE is used to filter rows before any grouping or aggregation takes place. It cannot be used with aggregate functions (like SUM, COUNT). HAVING is used to filter groups after the GROUP BY clause has been applied and can evaluate aggregate functions.

SYNTAX OVERVIEW

```
SELECT column1, SUM(column2)
FROM table
WHERE condition (Row-level)
GROUP BY column1
HAVING condition (Group-level);
```

REAL EXAMPLE

```
SELECT Department, COUNT(EmpID) as TotalStaff
FROM Employees
WHERE IsActive = 1
GROUP BY Department
HAVING COUNT(EmpID) > 5;
```

Output:

```
Department | TotalStaff
-----
Sales | 8
Engineering| 12
```

STEP-BY-STEP EXECUTION

- 1. The WHERE clause filters out inactive employees first.

- 2. The remaining rows are grouped by Department.
- 3. COUNT is calculated for each group.
- 4. HAVING filters out departments with 5 or fewer active employees.

Interview Tip

Always mention the Logical Order of Processing: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT.

Common Mistake

Trying to use aggregate functions in the WHERE clause (e.g., WHERE SUM(Sales) > 1000).

ADVANCED CONCEPTS

Real-World Use Case: Generating reports where you only want to highlight high-performing segments (e.g., Regions where total sales exceed \$100,000).

Best Practices: Filter as much data as possible in the WHERE clause before grouping to save computation power.

Performance Considerations: Filtering in WHERE utilizes indexes. Filtering in HAVING usually scans in-memory grouped data, which is slower if the initial dataset is huge.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can you use HAVING without GROUP BY?
- Does WHERE improve performance more than HAVING?

Quick Revision: Questions 6 to 10

Before moving forward, ensure you have mastered these key concepts. Interviewers often link these topics together during deep-dive rounds.

Memory Trick

Always map SQL concepts to real-life analogues. Joins are like Venn diagrams (but with tables). Indexes are like the glossary of a textbook. Transactions are like ATM withdrawals (all or nothing).

Interviewer's Checklist

- Did the candidate explain the 'Why' and not just the 'How'?
- Were performance and indexing mentioned voluntarily?
- Did they write syntax cleanly without prompt?

QUESTION 11

ACID Properties and Transactions

Difficulty: Advanced

Asked in: Infosys, Wipro, TCS, Amazon

WHY INTERVIEWERS ASK THIS

This evaluates your understanding of database constraints, data integrity, and relational design principles.

DETAILED ANSWER

A Primary Key uniquely identifies each record in a table; it cannot contain NULLs and a table can have only one. A Unique Key also ensures uniqueness but allows one NULL value, and a table can have multiple. A Foreign Key is a field in one table that uniquely identifies a row of another table, establishing a relationship and enforcing referential integrity.

SYNTAX OVERVIEW

```
CREATE TABLE Users (  
    ID INT PRIMARY KEY,  
    Email VARCHAR(50) UNIQUE,  
    DeptID INT FOREIGN KEY REFERENCES Depts(ID)  
);
```

REAL EXAMPLE

```
-- Creating constraints  
ALTER TABLE Orders  
ADD CONSTRAINT FK_User  
FOREIGN KEY (UserID) REFERENCES Users(ID);
```

Output:

Constraint applied successfully. Insertions failing referential integrity will now throw an error.

STEP-BY-STEP EXECUTION

- 1. Assign Primary Keys to surrogate IDs or unique identifiers.
- 2. Apply Unique constraints to business keys (Emails, Usernames).

- 3. Map relationships using Foreign Keys.

Interview Tip

Explain that Primary Keys automatically create a Clustered Index (in most RDBMS), while Unique Keys create a Non-Clustered Index.

Common Mistake

Confusing a Foreign Key with a simple integer column. Without the actual constraint, referential integrity is not enforced by the engine.

ADVANCED CONCEPTS

Real-World Use Case: Primary Key on UserID. Unique Key on Email to prevent duplicate sign-ups. Foreign Key on Order.UserID to ensure orders are placed by valid users.

Best Practices: Name your constraints explicitly (e.g., PK_Users, FK_Orders_Users) so they are easier to manage and drop later.

Performance Considerations: Always index Foreign Key columns, as they are frequently used in JOINS.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can a table have multiple foreign keys?
- What is a Composite Primary Key?
- What happens on DELETE CASCADE?

QUESTION 12

Deadlocks and Locks

Difficulty: Advanced

Asked in: Meta, Netflix, Amazon, Uber

WHY INTERVIEWERS ASK THIS

Window functions are critical for advanced analytics. Interviewers use this to see if you can perform complex data rankings and solve top-N per group problems.

DETAILED ANSWER

ROW_NUMBER() assigns a unique, sequential integer to rows. RANK() assigns the same rank to identical values but skips the next rank numbers (e.g., 1, 2, 2, 4). DENSE_RANK() assigns the same rank to identical values but does NOT skip numbers (e.g., 1, 2, 2, 3).

SYNTAX OVERVIEW

```
SELECT column,  
    ROW_NUMBER() OVER(ORDER BY column) as RN,  
    RANK() OVER(ORDER BY column) as Rnk,  
    DENSE_RANK() OVER(ORDER BY column) as DRnk  
FROM table;
```

REAL EXAMPLE

```
SELECT Score,  
    ROW_NUMBER() OVER(ORDER BY Score DESC) as RN,  
    RANK() OVER(ORDER BY Score DESC) as Rnk,  
    DENSE_RANK() OVER(ORDER BY Score DESC) as DRnk  
FROM ExamScores;
```

Output:

Score	RN	Rnk	DRnk
95	1	1	1
90	2	2	2
90	3	2	2
85	4	4	3

STEP-BY-STEP EXECUTION

- 1. The OVER clause defines the window (sorting by Score).
- 2. ROW_NUMBER blindly increments.
- 3. RANK gives ties the same value and jumps.
- 4. DENSE_RANK gives ties the same value but continues sequentially.

Interview Tip

If asked to find the 2nd highest salary, instantly mention DENSE_RANK().

Common Mistake

Using RANK() for Nth highest problems, which fails if multiple people share the 1st highest value.

ADVANCED CONCEPTS

Real-World Use Case: Finding the 'Nth Highest Salary' (Use DENSE_RANK to handle ties properly). Deduplicating rows (Use ROW_NUMBER with PARTITION BY).

Best Practices: Always specify an ORDER BY within the OVER() clause when using ranking functions, otherwise results are unpredictable.

Performance Considerations: Window functions require sorting, which can be memory intensive. Ensure indexes exist on the ORDER BY / PARTITION BY columns.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- How do you use PARTITION BY with these functions?
- How can ROW_NUMBER() be used to remove duplicates?

QUESTION 13

Views vs Materialized Views

Difficulty: Intermediate

Asked in: Microsoft, Oracle, Salesforce, Apple

WHY INTERVIEWERS ASK THIS

Understanding indexes is the difference between a query taking 1 second vs 1 hour. This tests your grasp on database architecture and optimization.

DETAILED ANSWER

A Clustered Index determines the physical order of data rows in a table (like a phone book sorted by last name); thus, a table can only have one. A Non-Clustered Index creates a separate structure that contains the indexed columns and a pointer to the actual data row (like an index at the back of a book); a table can have many.

SYNTAX OVERVIEW

```
CREATE CLUSTERED INDEX IX_EmpID ON Employees(EmpID);  
CREATE NONCLUSTERED INDEX IX_EmpName ON Employees(Name);
```

REAL EXAMPLE

```
-- Creating an index to speed up email lookups  
CREATE NONCLUSTERED INDEX idx_user_email  
ON Users (Email);
```

Output:

Index created. Subsequent SELECT queries filtering by Email will use an Index Seek instead of a Table Scan.

STEP-BY-STEP EXECUTION

- 1. The engine checks if an index exists for the WHERE clause.
- 2. For non-clustered, it traverses the B-Tree index, finds the pointer.
- 3. It uses the pointer (RowID or Clustered Key) to fetch the full row data.

Interview Tip

Mention B-Tree (Balanced Tree) architecture. Explain that primary keys create clustered indexes by default.

Common Mistake

Creating too many non-clustered indexes. Every index speeds up reads but slows down writes (INSERT/UPDATE/DELETE).

ADVANCED CONCEPTS

Real-World Use Case: Clustered index on ID columns (sequential). Non-clustered index on columns frequently used in WHERE, JOIN, or ORDER BY (e.g., Email, Status, CreatedDate).

Best Practices: Keep clustered index keys narrow, static, and ever-increasing (like an INT IDENTITY column).

Performance Considerations: Use non-clustered indexes for foreign keys to dramatically speed up JOIN operations.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- What is a Covering Index?
- What happens if you don't have a clustered index (Heap)?
- What is index fragmentation?

QUESTION 14

Triggers in SQL

Difficulty: Intermediate

Asked in: Stripe, Airbnb, Atlassian

WHY INTERVIEWERS ASK THIS

Interviewers want to see if you write clean, maintainable SQL code. CTEs are the modern standard for structuring complex queries.

DETAILED ANSWER

A Subquery is a query nested inside another query. A CTE is a temporary named result set defined using the WITH clause. CTEs are generally more readable, can be referenced multiple times within the same query, and support recursive queries, whereas subqueries can become deeply nested and harder to read.

SYNTAX OVERVIEW

```
WITH CTE_Name AS (  
    SELECT * FROM Table  
)  
SELECT * FROM CTE_Name;
```

REAL EXAMPLE

```
WITH HighEarnings AS (  
    SELECT EmpID, Salary FROM Employees WHERE Salary > 100000  
)  
SELECT * FROM HighEarnings WHERE EmpID IN (SELECT EmpID FROM Managers);
```

Output:

Returns all managers who also earn over 100,000.

STEP-BY-STEP EXECUTION

- 1. Define the temporary data set in the WITH clause.
- 2. The CTE is stored in memory during the query execution.
- 3. The main SELECT statement consumes the CTE data.

Interview Tip

If an interviewer asks how to query a self-referencing table (Manager-Employee), immediately say 'Recursive CTE'.

Common Mistake

Thinking CTEs are permanent or act like views. They only exist for the duration of the execution of the statement.

ADVANCED CONCEPTS

Real-World Use Case: Breaking down massive, 1000-line financial reporting queries into logical, sequential blocks. Traversing hierarchical data (like org charts) using Recursive CTEs.

Best Practices: Always use CTEs instead of derived tables (subqueries in the FROM clause) for better readability.

Performance Considerations: Performance between CTEs and standard subqueries is usually identical, as the query optimizer unpacks them the same way.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Are CTEs faster than Subqueries?
- How does a Recursive CTE work?
- Can you use multiple CTEs in one statement?

QUESTION 15

Correlated vs Non-Correlated Subqueries

Difficulty: Advanced

Asked in: Amazon, Flipkart, TCS, Infosys

WHY INTERVIEWERS ASK THIS

Interviewers ask this to test your understanding of DML vs DDL commands, transaction logging, and database memory management.

DETAILED ANSWER

DELETE is a DML command used to remove specific rows based on a WHERE clause. It logs each deletion and can be rolled back. TRUNCATE is a DDL command that removes all rows from a table by deallocating the data pages, making it much faster but it cannot be rolled back in some databases. DROP is a DDL command that completely removes the table structure, data, privileges, and indexes from the database.

SYNTAX OVERVIEW

```
DELETE FROM table_name WHERE condition;  
TRUNCATE TABLE table_name;  
DROP TABLE table_name;
```

REAL EXAMPLE

```
-- Delete specific row  
DELETE FROM Employees WHERE emp_id = 101;  
  
-- Empty the table  
TRUNCATE TABLE Employees;  
  
-- Destroy the table entirely  
DROP TABLE Employees;
```

Output:

```
DELETE: 1 row affected.  
TRUNCATE: Table emptied, 0 rows affected.  
DROP: Table no longer exists.
```

STEP-BY-STEP EXECUTION

- 1. Identify if you need to remove specific data (DELETE) or all data (TRUNCATE).
- 2. Use DROP only when the table structure is no longer needed.
- 3. Ensure you have backups before executing TRUNCATE or DROP.

Interview Tip

Always mention the difference in Transaction Logging (DELETE is row-level logged, TRUNCATE is page-level logged).

Common Mistake

Forgetting the WHERE clause in a DELETE statement, which accidentally removes all data (though much slower than TRUNCATE).

ADVANCED CONCEPTS

Real-World Use Case: Use DELETE when a user deletes their account. Use TRUNCATE to clear out daily staging tables in ETL processes. Use DROP when a temporary table is no longer needed after processing.

Best Practices: Always test DELETE statements as SELECT statements first to ensure you are targeting the correct rows.

Performance Considerations: TRUNCATE is significantly faster than DELETE without a WHERE clause because it doesn't write each deleted row to the transaction log.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can you roll back a TRUNCATE command?
- Does TRUNCATE reset the identity column/auto-increment?
- What permissions are required for DROP?

Quick Revision: Questions 11 to 15

Before moving forward, ensure you have mastered these key concepts. Interviewers often link these topics together during deep-dive rounds.

Memory Trick

Always map SQL concepts to real-life analogues. Joins are like Venn diagrams (but with tables). Indexes are like the glossary of a textbook. Transactions are like ATM withdrawals (all or nothing).

Interviewer's Checklist

- Did the candidate explain the 'Why' and not just the 'How'?
- Were performance and indexing mentioned voluntarily?
- Did they write syntax cleanly without prompt?

QUESTION 16

Normalization (1NF, 2NF, 3NF, BCNF)

Difficulty: Advanced

Asked in: Google, Uber, Microsoft, Swiggy

WHY INTERVIEWERS ASK THIS

Joins are the most fundamental aspect of relational databases. Interviewers want to know if you can correctly retrieve related data across multiple tables without data loss or duplication.

DETAILED ANSWER

INNER JOIN returns only the rows that have matching values in both tables. LEFT JOIN returns all rows from the left table, and matching rows from the right table (NULL if no match). RIGHT JOIN returns all rows from the right table, and matching rows from the left. FULL JOIN returns all rows when there is a match in either left or right table.

SYNTAX OVERVIEW

```
SELECT columns
FROM table1
[INNER | LEFT | RIGHT | FULL] JOIN table2
ON table1.column = table2.column;
```

REAL EXAMPLE

```
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
ON e.DeptID = d.DeptID;
```

Output:

```
Name | DepartmentName
-----
John | IT
Jane | NULL
```

STEP-BY-STEP EXECUTION

- 1. Identify the primary table (usually the one in the FROM clause).

- 2. Determine which tables need to be linked.
- 3. Choose the appropriate join type based on whether you need unmatched records.
- 4. Specify the linking condition in the ON clause.

Interview Tip

Venn diagrams are great, but explain Joins in terms of 'driving tables' and data preservation to impress senior interviewers.

Common Mistake

Using a LEFT JOIN but putting a filter on the right table in the WHERE clause, which accidentally converts it into an INNER JOIN.

ADVANCED CONCEPTS

Real-World Use Case: LEFT JOIN is heavily used to find missing relationships, e.g., finding all Users who have NOT placed an Order (where OrderID is NULL).

Best Practices: Always use table aliases (e.g., 'e' for Employees) to make queries readable and prevent ambiguous column errors.

Performance Considerations: Ensure the columns used in the ON clause are indexed. Avoid joining on text or highly unoptimized data types.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- What happens if you join tables with duplicate keys?
- What is a CROSS JOIN?
- How do you optimize a slow JOIN?

QUESTION 17

EXPLAIN and Query Execution Plans

Difficulty: Advanced

Asked in: Oracle, Adobe, Salesforce, Atlassian

WHY INTERVIEWERS ASK THIS

Tests the understanding of SQL's logical execution order. Many candidates confuse filtering raw data vs filtering aggregated data.

DETAILED ANSWER

WHERE is used to filter rows before any grouping or aggregation takes place. It cannot be used with aggregate functions (like SUM, COUNT). HAVING is used to filter groups after the GROUP BY clause has been applied and can evaluate aggregate functions.

SYNTAX OVERVIEW

```
SELECT column1, SUM(column2)
FROM table
WHERE condition (Row-level)
GROUP BY column1
HAVING condition (Group-level);
```

REAL EXAMPLE

```
SELECT Department, COUNT(EmpID) as TotalStaff
FROM Employees
WHERE IsActive = 1
GROUP BY Department
HAVING COUNT(EmpID) > 5;
```

Output:

```
Department | TotalStaff
-----
Sales | 8
Engineering | 12
```

STEP-BY-STEP EXECUTION

- 1. The WHERE clause filters out inactive employees first.

- 2. The remaining rows are grouped by Department.
- 3. COUNT is calculated for each group.
- 4. HAVING filters out departments with 5 or fewer active employees.

Interview Tip

Always mention the Logical Order of Processing: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT.

Common Mistake

Trying to use aggregate functions in the WHERE clause (e.g., WHERE SUM(Sales) > 1000).

ADVANCED CONCEPTS

Real-World Use Case: Generating reports where you only want to highlight high-performing segments (e.g., Regions where total sales exceed \$100,000).

Best Practices: Filter as much data as possible in the WHERE clause before grouping to save computation power.

Performance Considerations: Filtering in WHERE utilizes indexes. Filtering in HAVING usually scans in-memory grouped data, which is slower if the initial dataset is huge.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can you use HAVING without GROUP BY?
- Does WHERE improve performance more than HAVING?

QUESTION 18

Pagination in SQL

Difficulty: Intermediate

Asked in: Infosys, Wipro, TCS, Amazon

WHY INTERVIEWERS ASK THIS

This evaluates your understanding of database constraints, data integrity, and relational design principles.

DETAILED ANSWER

A Primary Key uniquely identifies each record in a table; it cannot contain NULLs and a table can have only one. A Unique Key also ensures uniqueness but allows one NULL value, and a table can have multiple. A Foreign Key is a field in one table that uniquely identifies a row of another table, establishing a relationship and enforcing referential integrity.

SYNTAX OVERVIEW

```
CREATE TABLE Users (  
  ID INT PRIMARY KEY,  
  Email VARCHAR(50) UNIQUE,  
  DeptID INT FOREIGN KEY REFERENCES Depts(ID)  
);
```

REAL EXAMPLE

```
-- Creating constraints  
ALTER TABLE Orders  
ADD CONSTRAINT FK_User  
FOREIGN KEY (UserID) REFERENCES Users(ID);
```

Output:

Constraint applied successfully. Insertions failing referential integrity will now throw an error.

STEP-BY-STEP EXECUTION

- 1. Assign Primary Keys to surrogate IDs or unique identifiers.
- 2. Apply Unique constraints to business keys (Emails, Usernames).

- 3. Map relationships using Foreign Keys.

Interview Tip

Explain that Primary Keys automatically create a Clustered Index (in most RDBMS), while Unique Keys create a Non-Clustered Index.

Common Mistake

Confusing a Foreign Key with a simple integer column. Without the actual constraint, referential integrity is not enforced by the engine.

ADVANCED CONCEPTS

Real-World Use Case: Primary Key on UserID. Unique Key on Email to prevent duplicate sign-ups. Foreign Key on Order.UserID to ensure orders are placed by valid users.

Best Practices: Name your constraints explicitly (e.g., PK_Users, FK_Orders_Users) so they are easier to manage and drop later.

Performance Considerations: Always index Foreign Key columns, as they are frequently used in JOINS.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- Can a table have multiple foreign keys?
- What is a Composite Primary Key?
- What happens on DELETE CASCADE?

QUESTION 19

EXISTS vs IN

Difficulty: Intermediate

Asked in: Meta, Netflix, Amazon, Uber

WHY INTERVIEWERS ASK THIS

Window functions are critical for advanced analytics. Interviewers use this to see if you can perform complex data rankings and solve top-N per group problems.

DETAILED ANSWER

ROW_NUMBER() assigns a unique, sequential integer to rows. RANK() assigns the same rank to identical values but skips the next rank numbers (e.g., 1, 2, 2, 4). DENSE_RANK() assigns the same rank to identical values but does NOT skip numbers (e.g., 1, 2, 2, 3).

SYNTAX OVERVIEW

```
SELECT column,  
    ROW_NUMBER() OVER(ORDER BY column) as RN,  
    RANK() OVER(ORDER BY column) as Rnk,  
    DENSE_RANK() OVER(ORDER BY column) as DRnk  
FROM table;
```

REAL EXAMPLE

```
SELECT Score,  
    ROW_NUMBER() OVER(ORDER BY Score DESC) as RN,  
    RANK() OVER(ORDER BY Score DESC) as Rnk,  
    DENSE_RANK() OVER(ORDER BY Score DESC) as DRnk  
FROM ExamScores;
```

Output:

Score	RN	Rnk	DRnk
95	1	1	1
90	2	2	2
90	3	2	2
85	4	4	3

STEP-BY-STEP EXECUTION

- 1. The OVER clause defines the window (sorting by Score).
- 2. ROW_NUMBER blindly increments.
- 3. RANK gives ties the same value and jumps.
- 4. DENSE_RANK gives ties the same value but continues sequentially.

Interview Tip

If asked to find the 2nd highest salary, instantly mention DENSE_RANK().

Common Mistake

Using RANK() for Nth highest problems, which fails if multiple people share the 1st highest value.

ADVANCED CONCEPTS

Real-World Use Case: Finding the 'Nth Highest Salary' (Use DENSE_RANK to handle ties properly). Deduplicating rows (Use ROW_NUMBER with PARTITION BY).

Best Practices: Always specify an ORDER BY within the OVER() clause when using ranking functions, otherwise results are unpredictable.

Performance Considerations: Window functions require sorting, which can be memory intensive. Ensure indexes exist on the ORDER BY / PARTITION BY columns.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- How do you use PARTITION BY with these functions?
- How can ROW_NUMBER() be used to remove duplicates?

QUESTION 20

SQL Injection and Prevention

Difficulty: Beginner

Asked in: Microsoft, Oracle, Salesforce, Apple

WHY INTERVIEWERS ASK THIS

Understanding indexes is the difference between a query taking 1 second vs 1 hour. This tests your grasp on database architecture and optimization.

DETAILED ANSWER

A Clustered Index determines the physical order of data rows in a table (like a phone book sorted by last name); thus, a table can only have one. A Non-Clustered Index creates a separate structure that contains the indexed columns and a pointer to the actual data row (like an index at the back of a book); a table can have many.

SYNTAX OVERVIEW

```
CREATE CLUSTERED INDEX IX_EmpID ON Employees(EmpID);  
CREATE NONCLUSTERED INDEX IX_EmpName ON Employees(Name);
```

REAL EXAMPLE

```
-- Creating an index to speed up email lookups  
CREATE NONCLUSTERED INDEX idx_user_email  
ON Users (Email);
```

Output:

Index created. Subsequent SELECT queries filtering by Email will use an Index Seek instead of a Table Scan.

STEP-BY-STEP EXECUTION

- 1. The engine checks if an index exists for the WHERE clause.
- 2. For non-clustered, it traverses the B-Tree index, finds the pointer.
- 3. It uses the pointer (RowID or Clustered Key) to fetch the full row data.

Interview Tip

Mention B-Tree (Balanced Tree) architecture. Explain that primary keys create clustered indexes by default.

Common Mistake

Creating too many non-clustered indexes. Every index speeds up reads but slows down writes (INSERT/UPDATE/DELETE).

ADVANCED CONCEPTS

Real-World Use Case: Clustered index on ID columns (sequential). Non-clustered index on columns frequently used in WHERE, JOIN, or ORDER BY (e.g., Email, Status, CreatedDate).

Best Practices: Keep clustered index keys narrow, static, and ever-increasing (like an INT IDENTITY column).

Performance Considerations: Use non-clustered indexes for foreign keys to dramatically speed up JOIN operations.

FOLLOW-UP QUESTIONS TO ANTICIPATE

- What is a Covering Index?
- What happens if you don't have a clustered index (Heap)?
- What is index fragmentation?

Quick Revision: Questions 16 to 20

Before moving forward, ensure you have mastered these key concepts. Interviewers often link these topics together during deep-dive rounds.

Memory Trick

Always map SQL concepts to real-life analogues. Joins are like Venn diagrams (but with tables). Indexes are like the glossary of a textbook. Transactions are like ATM withdrawals (all or nothing).

Interviewer's Checklist

- Did the candidate explain the 'Why' and not just the 'How'?
- Were performance and indexing mentioned voluntarily?
- Did they write syntax cleanly without prompt?

SQL Cheat Sheets

Essential Window Functions

Function	Description	Use Case
ROW_NUMBER ()	Unique number for each row	Pagination, Deduplication
RANK ()	Ranks with gaps for ties	Top-N leaderboards
DENSE_RANK ()	Ranks without gaps for ties	Nth Highest Salary
LEAD () / LAG ()	Accesses next/previous row	Year-over-Year comparisons

String & Date Functions

Function	Syntax	Example Output
SUBSTRING	SUBSTRING(col, start, len)	Extracting domain from email
COALESCE	COALESCE(col1, col2, 'default')	Replacing NULLs efficiently
DATEDIFF	DATEDIFF(day, date1, date2)	Calculating days since signup

Thank You.

Keep Learning. Keep Building. Keep Growing.

About AlgoTutor

AlgoTutor is the premier destination for tech career growth. We specialize in providing industry-standard interview preparation, System Design roadmaps, DSA strategies, and comprehensive guides for Backend, AI, and Full Stack development. Our mission is to bridge the gap between foundational knowledge and production engineering.

"Every SQL query you master brings you one step closer to your dream job."

Connect with us on: [LinkedIn](#) | [GitHub](#) | [YouTube](#) | [Discord](#)
www.algotutor.io